ROS 2 Humble on Georgia Tech PACE

Setup, upload, custom images, building, running, and Slurm

PACE login

```
ssh GT_USERNAME@login-phoenix.pace.gatech.edu
```

Project settings

```
PROJECT="$HOME/r-mhale30-0"
WORKSPACE="$PROJECT/ros2_humble_ws"
CONTAINERS="$PROJECT/containers"
```

This guide uses allocation `gts-mhale30`, partition `cpu-small`, and ROS 2 Humble on Ubuntu 22.04.

Text in monospaced blocks is intended to be copied. Replace placeholders such as `GT_USERNAME`, `JOB_ID`, and package names.

Revised September 2026

1. The PACE workflow

PACE separates job management from computation. This explains why Apptainer was not found on the login node but appeared after requesting a compute node.

- Login node: edit files, upload code, inspect logs, submit jobs, and check the queue.
- Compute node: run Apptainer, compile packages, and run ROS experiments.

Submitting `sbatch job.sh` from the login node is correct. Slurm executes it on a compute node. Do not run long builds or ROS experiments directly on the login node.

Connect from the coworker's computer

```
ssh GT_USERNAME@login-phoenix.pace.gatech.edu
```

Replace `GT_USERNAME` with the coworker's Georgia Tech username and complete authentication.

Request an interactive compute node for setup and debugging

```
srun \
  --account=gts-mhale30 \
  --partition=cpu-small \
  --nodes=1 \
  --ntasks=1 \
  --cpus-per-task=4 \
  --mem=8G \
  --time=02:00:00 \
  --pty bash
```

```
hostname
command -v apptainer
apptainer --version
```

On the tested Phoenix compute node, Apptainer was available as `/usr/bin/apptainer`.

2. Persistent project layout

Keep images, source, datasets, configuration, results, and logs under project storage. A SIF image is reusable and normally read-only; code and results live outside it.

```
PROJECT="$HOME/r-mhale30-0"
WORKSPACE="$PROJECT/ros2_humble_ws"
CONTAINERS="$PROJECT/containers"

mkdir -p "$CONTAINERS"
mkdir -p "$WORKSPACE/src"
mkdir -p "$WORKSPACE/datasets"
mkdir -p "$WORKSPACE/config"
mkdir -p "$WORKSPACE/results"
mkdir -p "$WORKSPACE/slurm_logs"
```

Recommended layout

- `containers/` - reusable SIF images and definition files.
- `ros2_humble_ws/src/` - the ROS packages.
- `ros2_humble_ws/datasets/` - input datasets.
- `ros2_humble_ws/results/` - persistent output.
- `build/`, `install/`, and `log/` - created by colcon.

What is a .sif file?

A Singularity Image Format file is Apptainer's portable container image. It holds the operating system, ROS distribution, libraries, and tools. Multiple Slurm jobs may safely read the same completed SIF at once.

Do not put changing data inside the image: Bind the workspace into the container. Then files written under `/workspace` persist in the host workspace.

3. Choose an official ROS image

Start with the smallest image that already contains the needed ROS layer. If the project needs extra Ubuntu or ROS dependencies, derive one custom SIF from that base.

ROS Core - minimal ROS communication tools: `docker://ros:humble-ros-core`

ROS Base - headless nodes, building, and common CLI tools: `docker://ros:humble-ros-base`

Perception - base plus perception-related packages: `docker://ros:humble-perception`

Desktop - RViz and GUI-oriented packages: `docker://osrf/ros:humble-desktop`

Official links

- Official ROS Docker image: https://hub.docker.com/_/ros
- Official Humble tag search: https://hub.docker.com/_/ros/tags?name=humble
- OSRF ROS images: https://hub.docker.com/r/osrf/ros

Images are layered choices, not plug-ins: Do not try to combine several SIF files. Select one suitable base image, then add any missing dependencies to one custom image.

Pull and reuse named images

```
BASE_IMAGE="$CONTAINERS/ros_humble_base.sif"
PERCEPTION_IMAGE="$CONTAINERS/ros_humble_perception.sif"

apptainer pull "$BASE_IMAGE" docker://ros:humble-ros-base
apptainer pull "$PERCEPTION_IMAGE" docker://ros:humble-perception
```

Pull only once on a compute node. Later jobs point `IMAGE` at the desired SIF. Give each image a descriptive, versioned name. Do not have two jobs build or overwrite the same SIF simultaneously.

4. Put the ROS project in the workspace

The final location should be `$HOME/r-mhale30-0/ros2_humble_ws/src`. Use whichever transfer method is most convenient.

Option 1: GitHub

Use this when the project is already in a Git repository. Run on the PACE login node:

```
cd "$HOME/r-mhale30-0/ros2_humble_ws/src"
git clone REPOSITORY_URL
```

For a private repository, use the approved GitHub authentication method for the account.

Option 2: rsync or scp

Run these from the coworker's local computer, not from PACE. `rsync` is convenient for repeated updates:

```
rsync -av --progress /local/path/to/src/ \
  GT_USERNAME@login-phoenix.pace.gatech.edu:~/r-mhale30-0/ros2_humble_ws/src/
```

For a simple one-time copy:

```
scp -r /local/path/to/src \
  GT_USERNAME@login-phoenix.pace.gatech.edu:~/r-mhale30-0/ros2_humble_ws/
```

Option 3: Globus

Globus is useful for large datasets or large directory trees. Open https://app.globus.org/, sign in through Georgia Tech, select the PACE collection made available to the account, and transfer the files into the workspace. If the correct collection is unclear, use the collection name listed in the current PACE documentation or ask PACE support.

Option 4: PACE web dashboard

The PACE web dashboard (Open OnDemand) has a Files browser with an Upload button. Sign in to the dashboard supplied by PACE, browse to `r-mhale30-0/ros2_humble_ws/src`, and upload the project. This is easiest for small transfers but is less convenient for very large datasets.

Verify on PACE

```
find "$HOME/r-mhale30-0/ros2_humble_ws/src" \
  -maxdepth 3 -name package.xml -print
```

Correct placement: A package should look like `$WORKSPACE/src/dpgo/package.xml`, not like `$WORKSPACE/src/src/dpgo/package.xml` unless the uploaded project intentionally has that nesting.

Check dependencies before building

- Read the package README for system setup and dataset instructions.
- Inspect `package.xml` for ROS dependencies.
- Inspect `CMakeLists.txt` for `find_package(...)` calls.
- Check Python imports and any `requirements.txt` file.

5. Build and test in the selected image

Start an interactive shell from a compute node. `--cleanenv` prevents PACE compiler and module variables from leaking into Ubuntu inside the container.

```
IMAGE="$HOME/r-mhale30-0/containers/ros_humble_base.sif"
WORKSPACE="$HOME/r-mhale30-0/ros2_humble_ws"

apptainer shell \
  --cleanenv \
  --bind "$WORKSPACE:/workspace" \
  "$IMAGE"
```

Inside the `Apptainer>` prompt:

```
source /opt/ros/humble/setup.bash
cd /workspace

command -v gcc
command -v g++
gcc --version

colcon build --parallel-workers 4
```

After a successful build:

```
source /workspace/install/setup.bash
ros2 pkg executables dpgo
ros2 launch dpgo dpgo_multirobot.launch.py
```

For a single executable, use:

```
ros2 run PACKAGE_NAME EXECUTABLE_NAME
```

Why --cleanenv mattered in the test: Without it, host compiler settings leaked into the container and the C++ build could not find `stddef.h`. With `--cleanenv`, GCC 11 and the C headers inside Ubuntu worked correctly.

Rebuild one package cleanly after an image or dependency change

```
rm -rf /workspace/build/dpgo /workspace/install/dpgo
cd /workspace
colcon build --packages-select dpgo --parallel-workers 4 --cmake-clean-cache
```

Delete only the exact package build/install directories. Do not erase the entire workspace unless that is intentional.

6. Build a custom image when dependencies are missing

A custom Apptainer definition file records every required package. This is more reproducible than manually modifying a running container.

Example definition file: ros_humble_dpgo.def

```
Bootstrap: docker
From: ros:humble-perception

%post
    apt-get update
    DEBIAN_FRONTEND=noninteractive apt-get install -y \
        libsuitesparse-dev \
        libgoogle-glog-dev \
        libunwind8 \
        libunwind-dev
    apt-get clean
    rm -rf /var/lib/apt/lists/*

%environment
    export ROS_DISTRO=humble
```

Build it on an interactive compute node

```
PROJECT="$HOME/r-mhale30-0"
DEF_FILE="$PROJECT/containers/ros_humble_dpgo.def"
IMAGE="$PROJECT/containers/ros_humble_dpgo.sif"

apptainer build --fakeroot --force "$IMAGE" "$DEF_FILE"
```

Custom-image references

- Apptainer definition files: https://apptainer.org/docs/user/main/definition_files.html
- Building images: https://apptainer.org/docs/user/main/build_a_container.html
- Using Docker and OCI images: https://apptainer.org/docs/user/main/docker_and_oci.html

If --fakeroot is not authorized: Stop and ask PACE support about the supported image-build workflow, or build the image in an approved environment and upload the completed SIF. Do not improvise around cluster permissions.

Dependency errors seen in this project

- `SPQR_INCLUDES` or `SPQR_LIBRARIES` missing: add `libsuitesparse-dev`.
- `glog/logging.h` missing: add `libgoogle-glog-dev`.
- `libunwind.so.8` missing: add `libunwind8` and `libunwind-dev`.

The CHOLMOD package-name mismatch shown as a yellow CMake developer warning was not the fatal error; the red missing-library messages were.

7. Verify the custom image before a long job

First rebuild the package using the same custom image that will run it. Then check the final executable for unresolved shared libraries.

```
IMAGE="$HOME/r-mhale30-0/containers/ros_humble_dpgo.sif"
WORKSPACE="$HOME/r-mhale30-0/ros2_humble_ws"

apptainer exec \
  --cleanenv \
  --bind "$WORKSPACE:/workspace" \
  "$IMAGE" \
  bash -c '
    set -e
    source /opt/ros/humble/setup.bash
    cd /workspace
    rm -rf build/dpgo install/dpgo
    colcon build --packages-select dpgo --parallel-workers 4
    source install/setup.bash
    ldd install/dpgo/lib/dpgo/dpgo_robot_node | grep "not found" && exit 1 || true
    ros2 pkg executables dpgo
  '
```

About the ldd check: No `not found` lines is the desired result. The command above deliberately fails if an unresolved library is found.

A simpler interactive check

```
ldd /workspace/install/dpgo/lib/dpgo/dpgo_robot_node \
  | grep "not found" || echo "All shared libraries found"
```

Keep the image and workspace paired

C++ binaries link against libraries available during the build. If the image changes, rebuild affected packages with that image before submitting production jobs. Store the exact image path in each Slurm script.

8. Recommended dataset interface: ROS 2 parameters

Use ROS terms: The usual ROS 2 mechanisms are parameters for node configuration and launch arguments for values exposed by a launch file. Prefer these over hard-coded dataset and output paths.

Why this is better

- Build the package once, then run any number of datasets without editing or recompiling code.
- Each Slurm job records its dataset and output directory in the submission command.
- Parallel jobs can use different output directories and ROS domain IDs without duplicating source trees.

C++ node pattern

```
this->declare_parameter<std::string>("dataset_path", "");
this->declare_parameter<std::string>("output_dir", "");

const auto dataset_path =
  this->get_parameter("dataset_path").as_string();
const auto output_dir =
  this->get_parameter("output_dir").as_string();
```

Run a node directly with parameters

```
ros2 run dpgo dpgo_robot_node --ros-args \
  -p dataset_path:=/workspace/datasets/M3500 \
  -p output_dir:=/workspace/results/M3500
```

Run a launch file with exposed launch arguments

```
ros2 launch dpgo dpgo_multirobot.launch.py \
  dataset_path:=/workspace/datasets/M3500 \
  output_dir:=/workspace/results/M3500
```

The launch file must declare these arguments and pass them to the node parameters. Names are package-specific; do not assume a launch file accepts an argument until it declares it.

9. Discover arguments and use parameter files

Show the launch arguments that a launch file actually accepts:

```
ros2 launch dpgo dpgo_multirobot.launch.py --show-args
```

Inspect parameters on a running node:

```
ros2 node list
ros2 param list /dpgo_robot_node_0
ros2 param describe /dpgo_robot_node_0 dataset_path
ros2 param get /dpgo_robot_node_0 dataset_path
```

Parameter YAML example: config/M3500.yaml

```
/**:
  ros__parameters:
    dataset_path: /workspace/datasets/M3500
    output_dir: /workspace/results/M3500
```

The wildcard applies values to all matching nodes. Use a specific node name instead when robots need different values.

Direct node with YAML

```
ros2 run dpgo dpgo_robot_node --ros-args \
  --params-file /workspace/config/M3500.yaml
```

For a launch file, either pass the YAML path as a launch argument (if the launch file supports one) or modify the launch file once so its Node actions load the YAML.

Best project change: Expose both `dataset_path` and `output_dir`. The exact names can differ, but they should be command-line configurable and visible in `--show-args` or the node parameter list.

ROS 2 documentation

- Parameters: ROS 2 Humble parameter tutorial
- Node CLI arguments: Passing ROS arguments to nodes
- Launch substitutions and arguments: Using substitutions

10. Reusable Slurm script for one dataset

Save as `run_dpgo.sh`. It accepts the dataset path, output path, and optional ROS domain ID as arguments to `sbatch`.

```
#!/bin/bash
#SBATCH --job-name=dpgo
#SBATCH --account=gts-mhale30
#SBATCH --partition=cpu-small
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --mem=8G
#SBATCH --time=1-00:00:00
#SBATCH --output=slurm_logs/dpgo_%j.out
#SBATCH --error=slurm_logs/dpgo_%j.err
#SBATCH --mail-type=END,FAIL
#SBATCH --mail-user=GT_USERNAME@gatech.edu

set -euo pipefail

PROJECT="$HOME/r-mhale30-0"
IMAGE="$PROJECT/containers/ros_humble_dpgo.sif"
WORKSPACE="$PROJECT/ros2_humble_ws"

DATASET_PATH="${1:?Usage: sbatch run_dpgo.sh DATASET OUTPUT [DOMAIN_ID]}"
OUTPUT_DIR="${2:?Usage: sbatch run_dpgo.sh DATASET OUTPUT [DOMAIN_ID]}"
DOMAIN_ID="${3:-51}"

mkdir -p "$WORKSPACE/slurm_logs"
mkdir -p "$WORKSPACE/results"

test -f "$IMAGE"
test -f "$WORKSPACE/install/setup.bash"

apptainer exec \
  --cleanenv \
  --env "ROS_DOMAIN_ID=$DOMAIN_ID" \
  --env "ROS_LOG_DIR=$OUTPUT_DIR/ros_logs" \
  --bind "$WORKSPACE:/workspace" \
  "$IMAGE" \
  bash -c "
    set -e
    source /opt/ros/humble/setup.bash
    source /workspace/install/setup.bash
    mkdir -p '$OUTPUT_DIR' '$OUTPUT_DIR/ros_logs'
    ros2 launch dpgo dpgo_multirobot.launch.py \
      dataset_path:='$DATASET_PATH' \
      output_dir:='$OUTPUT_DIR'
  "
```

Use container-visible paths such as `/workspace/datasets/...`. Because `/workspace` is bound to the persistent host workspace, results remain after the job ends.

11. Submit one or several datasets

From the workspace on the login node:

```
cd "$HOME/r-mhale30-0/ros2_humble_ws"
mkdir -p slurm_logs results
chmod +x run_dpgo.sh
```

One dataset

```
sbatch run_dpgo.sh \
  /workspace/datasets/M3500 \
  /workspace/results/M3500 \
  51
```

Two datasets in parallel

```
sbatch run_dpgo.sh \
  /workspace/datasets/M3500 \
  /workspace/results/M3500 \
  51

sbatch run_dpgo.sh \
  /workspace/datasets/M3500_variant \
  /workspace/results/M3500_variant \
  52
```

Slurm assigns separate job IDs. Both jobs reuse the same SIF and installed package, while writing to different result directories. Different `ROS_DOMAIN_ID` values isolate ROS discovery traffic.

Avoid conflicts: Parallel jobs must not write to the same output files. Also do not run `colcon build` concurrently in the same workspace. Build first, then submit read-only runs.

Fallback when the dataset is still hard-coded

For sequential jobs only: edit the dataset and output paths, build, submit, wait for completion, then edit/build/submit the second version. Once a job starts, it normally runs the already-built executable, so later source edits do not alter that running process.

For concurrent hard-coded variants, use separate workspace copies with separate `build`, `install`, and result directories. This works, but parameters are cleaner and safer.

12. Slurm commands: submit, monitor, cancel

Submit a job:

```
sbatch run_dpgo.sh DATASET OUTPUT DOMAIN_ID
```

Show your queued and running jobs:

```
squeue -u "$USER"
```

Show one active job:

```
squeue -j JOB_ID
```

Cancel one job:

```
scancel JOB_ID
```

Cancel all of your jobs; use this carefully:

```
scancel -u "$USER"
```

Know whether a vanished job finished

```
sacct -j JOB_ID \
  --format=JobID,JobName,State,ExitCode,Elapsed,MaxRSS,ReqMem
```

- COMPLETED and 0:0: successful Slurm completion.
- FAILED: inspect the error log and exit code.
- OUT_OF_MEMORY: request more memory.
- TIMEOUT: increase the time request or optimize the job.
- CANCELLED: the job was cancelled by a user or system action.

Email notification

The Slurm script includes `#SBATCH --mail-type=END,FAIL` and `#SBATCH --mail-user=...`. Replace the placeholder email before submission. Email is useful, but `sacct` remains the authoritative check.

If `seff` is unavailable on PACE, use `sacct` as shown above.

13. Read output, errors, and resource use

The example creates one standard-output and one standard-error file per job ID.

```
ls -lh slurm_logs/
tail -n 100 slurm_logs/dpgo_JOB_ID.out
tail -n 100 slurm_logs/dpgo_JOB_ID.err
tail -f slurm_logs/dpgo_JOB_ID.out
grep -Ei "error|failed|exception|killed|not found" \
  slurm_logs/dpgo_JOB_ID.err
```

Stop `tail -f` with Ctrl-C; it does not cancel the job.

Check partitions and availability

```
sinfo
sinfo -p cpu-small
scontrol show partition cpu-small
squeue -p cpu-small
```

Is cpu-small enough?

The partition name selects a class of resources; `--cpus-per-task`, `--mem`, and `--time` set the job request. Begin with 4 CPUs and 8 GB for a modest CPU-only ROS job, then inspect actual use.

```
sacct -j JOB_ID \
  --format=JobID,State,Elapsed,AllocCPUS,ReqMem,MaxRSS,ExitCode
```

- If MaxRSS is close to requested memory, request more next time.
- If CPU use is low and the application is not parallel, additional CPUs will not speed it up.
- If a package needs CUDA/GPU, use an appropriate GPU partition and a CUDA-capable image; `cpu-small` is not a GPU resource.
- Ask PACE documentation/support about account-specific partitions, limits, and QOS when `sinfo` does not tell the full policy.

14. Troubleshooting checklist

Apptainer is not found on the login node: request a compute node with `srun`, or submit the work with `sbatch`.

stddef.h is missing during a C++ build: use `--cleanenv` and rebuild with the compiler inside the image.

The ROS package is not found: fix the earlier build error, rebuild, and source `install/setup.bash`.

SPQR, glog, or libunwind is missing: add the required Ubuntu package to the definition file, rebuild the SIF, and rebuild the ROS package.

Output disappeared: write it under `/workspace/results`, which is bound to persistent storage.

Parallel jobs interfere: use different output directories and ROS domain IDs. Do not build concurrently in the same workspace.

A job vanished from squeue: check `sacct -j JOB_ID` and read its output and error logs.

About set -euo pipefail

- `-e`: stop the script when a command fails.
- `-u`: stop when an unset variable is used.
- `pipefail`: make a pipeline fail when any command in it fails.

This prevents a failed setup, build, or launch prerequisite from being silently followed by misleading commands.

15. Quick-start sequence

One-time actions

- SSH to `GT_USERNAME@login-phoenix.pace.gatech.edu`.
- Request an interactive compute node and verify Apptainer.
- Create the project directories and upload the complete `src` directory.
- Pull an official image or build one custom image containing every dependency.
- Build the workspace inside that exact image with `--cleanenv`.
- Run the ldd check and a short ROS smoke test.

For every dataset

- Put the input under `/workspace/datasets` and choose a unique results directory.
- Submit `run_dpgo.sh` with dataset, output, and domain-ID arguments.
- Monitor with `squeue -u $USER`; verify completion with `sacct`.
- Inspect both the Slurm logs and the saved results directory.

Final recommendation

Configure; do not duplicate: Keep one source tree and one tested image. Make dataset path, output directory, robot count, thresholds, and similar experiment choices ROS 2 parameters or launch arguments. Duplicate workspaces only as a temporary fallback for code that cannot yet be configured.

Primary references

- Apptainer Docker/OCI support: apptainer.org/docs/user/main/docker_and_oci.html
- Apptainer definition files: apptainer.org/docs/user/main/definition_files.html
- Apptainer bind paths: apptainer.org/docs/user/main/bind_paths_and_mounts.html
- ROS 2 Humble documentation: docs.ros.org/en/humble